Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
по дисциплине «Технологии и инструментальные средства
проектирования интеллектуальных систем»
на тему:

ИГРОВОЙ АССИСТЕНТ

БГУИР КР 6-05-0611-03 022 ПЗ

Студент гр. 321703 Р. Д. Давыдов
Проверяющий С. А. Самодумкин

Минск 2026

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет ИТиУ Кафедра ИИТ
Специальность 6-05-0611-03

УТВЕРЖДАЮ

_____ Зав. кафедрой
«___» _____ 2026 г.

ЗАДАНИЕ
по курсовой работе студента
Давыдов Рената Дмитриевича
(фамилия, имя, отчество)

1. Тема работы: Игровой ассистент

2. Срок сдачи студентом законченной работы: 12.12.2026

3. Исходные данные к проекту: `https://github.com/Retinnel/Minecraft_AI_Agent`

4. Содержание пояснительной записки (перечень подлежащих разработке вопросов): Введение
1 Анализ подходов к разработке игрового ассистента.
2 Проектирование игрового ассистента.
3 Разработка игрового ассистента.
Заключение

5. Перечень графического материала (с точным указанием обязательных чертежей): Компьютерная презентация курсового проекта _____

КАЛЕНДАРНЫЙ ПЛАН

№ п/п	Наименование этапов курсовой работы	Объем этапа, %	Срок выполнения этапов	Примечание
1	Подбор и изучение литературы	10	12.09 – 20.09	
2	Изучение проблемной области, средств проектирования и разработки	10	21.09 – 27.09	
3	Определение требований к системе	10	28.09 – 06.10	
4	Проектирование системы	25	07.10 – 13.11	
5	Реализация системы	30	14.11 – 12.12	
6	Оформление пояснительной записки	10	25.11 – 12.12	
7	Оформление графической части проекта	5	01.12 – 12.12	

Дата выдачи задания: 12.09.2026 г. Куратор _____ С. А. Самодумкин

Задание принял к исполнению _____ Р. Д. Давыдов

СОДЕРЖАНИЕ

Перечень условных обозначений 4
Введение 5
1 Анализ предметной области и формирование требований к гибридной интеллектуальной системе 7
1.1 Характеристика предметной области 7
1.2 Анализ решаемого класса задач 8
1.3 Анализ базы знаний, решателя задач и интерфейса 9
1.4 Анализ существующих гибридных интеллектуальных систем 11
1.5 Требования к гибридной интеллектуальной системе 12
1.6 Выводы 13
2 Проектирование гибридной интеллектуальной системы 15
2.1 Определение пользователей гибридной интеллектуальной системы 15
2.2 Сценарии использования гибридной интеллектуальной системы 16
2.3 Общая архитектура гибридной интеллектуальной системы . 19
2.4 Проектирование алгоритмов и методов решения задач 22
2.5 Выводы 27
3 Реализация, тестирование и демонстрация гибридной интеллектуальной системы 29
3.1 Выбор средств и технологий реализации 29
3.2 Структура программной реализации 29
3.3 Реализация пользовательского интерфейса 29
3.4 Тестирование интеллектуальной системы 29
3.5 Оценка качества, эффективности и объяснимости системы . 29
3.6 Демонстрация работы гибридной интеллектуальной системы 29
3.7 Анализ ограничений и направления развития 29
3.8 Выводы 29
Заключение 30
Список использованных источников 32

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

БЗ – база знаний;

ГИС – гибридная интеллектуальная система;

ПрО – предметная область;

API (Application Programming Interface) – программный интерфейс приложения;

JSON (JavaScript Object Notation) – текстовый формат обмена данными;

LLM (Large Language Model) – большая языковая модель;

OWL (Web Ontology Language) – язык описания онтологий;

RAG (Retrieval-Augmented Generation) – дополнение генерации извлечением;

RDF (Resource Description Framework) – модель представления данных в виде троек «субъект – предикат – объект».

ВВЕДЕНИЕ

Большие языковые модели сделали возможным управление программными системами на естественном языке, что особенно органично для интерактивных виртуальных сред. Однако у ассистентов, построенных исключительно на языковой модели, обнаруживается принципиальное ограничение: модель хранит знания в весах, полученных при обучении, и не располагает ни актуальным состоянием среды, ни механизмом проверки собственных утверждений. Как следствие, она порождает правдоподобные, но неверные идентификаторы объектов и не способна гарантировать соблюдение ограничений. Ошибка при этом совершается молча: внешне ответ неотличим от корректного, и пользователь не получает сигнала о том, что его поняли неправильно[1].

Разрешение противоречия достигается объединением двух подходов в единой архитектуре, при котором логико-символьный подход отвечает за хранение знаний, вывод и объяснение, а нейробионический – за понимание свободных формулировок. Системы такого рода называются гибридными и рассматриваются как одно из основных направлений развития искусственного интеллекта. Актуальность темы определяется тем, что вынесение знаний и проверок за пределы языковой модели даёт гарантии корректности, недостижимые статистическими методами, при сохранении естественности взаимодействия. Задача возникает всюду, где языковая модель управляет действиями в системе с собственным состоянием и собственными правилами[2].

Целью курсовой работы является разработка гибридной интеллектуальной системы игрового ассистента, объединяющей онтологическую базу знаний, решатель задач и большую языковую модель в едином процессе решения задач пользователя.

Для достижения цели необходимо решить следующие задачи:

– проанализировать предметную область, класс решаемых задач и существующие гибридные интеллектуальные системы;

– определить состав и назначение компонентов системы: базы знаний, решателя задач и интерфейса;

– сформировать требования к гибридной интеллектуальной системе;

– спроектировать архитектуру, определить точки интеграции компонентов и стратегию их совместного применения;

– реализовать базу знаний, решатель задач и интеграцию компонентов;

– провести тестирование и сравнительную оценку гибридного, чисто символьного и чисто нейросетевого вариантов.

Логико-символьный подход представлен онтологической базой знаний в формате RDF/OWL, правилами ограничений и решателем, выполня-

ющим обратный вывод по графу способов получения объектов; нейробионический – большой языковой моделью, которая извлекает намерение из свободной реплики, подбирает официальные наименования для разговорных и формулирует ответ по выходу решателя.

Связь с предшествующими этапами курсового проектирования состоит в следующем: интерфейс – клиент виртуальной среды и серверная часть – унаследован от курсового проекта 6-го семестра и расширен; база знаний и решатель задач разработаны заново в соответствии с требованиями к курсовым проектам 4-го и 5-го семестров, поскольку предметная область требует онтологического представления знаний и вывода по графу способов получения объектов.

Объектом исследования являются гибридные интеллектуальные системы, объединяющие логико-символьные и нейросетевые методы. Предметом исследования являются методы интеграции онтологической базы знаний и решателя задач с большой языковой моделью в диалоговой системе управления действиями.

Практическая значимость работы состоит в том, что предложенная архитектура обеспечивает нулевую долю недопустимых действий по построению, а не по статистике: решение о допустимости принимается решателем после языковой модели и не может быть изменено формулировкой запроса.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ГИБРИДНОЙ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЕ

1.1 Характеристика предметной области

Предметной областью разрабатываемой системы является взаимодействие пользователя с интерактивной виртуальной средой посредством естественного языка. В качестве конкретной среды выбрана Minecraft версии 1.20.1 – песочница с открытым миром, широко применяемая в исследованиях интеллектуальных агентов в качестве испытательного полигона. Выбор обусловлен тремя обстоятельствами: среда обладает формализованными и доступными для машинной обработки данными о своих объектах, допускает программное расширение, а поведение пользователя в ней не предопределено сценарием, что обеспечивает разнообразие запросов[3, 4].

Основными сущностями предметной области являются:

– предметы – объекты, которые могут находиться в инвентаре пользователя и расходоваться при изготовлении;

– блоки – объекты, размещаемые в мире и добываемые из него;

– существа – подвижные объекты мира, из которых при победе над ними извлекаются предметы;

– рецепты – способы получения одних предметов из других с указанием состава, количества и требуемого оборудования;

– группы – именованные множества взаимозаменяемых предметов, позволяющие задавать состав рецепта обобщённо;

– инструменты – предметы, наличие которых необходимо для добычи определённых блоков, характеризуемые видом и уровнем материала.

Между сущностями определены отношения: предмет входит в состав рецепта, рецепт производит предмет, блок даёт предмет при добыче, блок требует инструмента определённого уровня, группа содержит предметы. Совокупность этих отношений образует ориентированный граф способов получения объектов, обход которого составляет основу решения задач в разрабатываемой системе.

Существенной особенностью предметной области является наличие состояния пользователя: содержимого инвентаря, имеющихся инструментов, показателей здоровья и параметров окружения. Одна и та же формулировка запроса при разном состоянии требует разного ответа, поэтому знания о мире и сведения о текущем состоянии должны использоваться совместно.

Источником знаний служат штатные файлы данных среды, описы-

вающие рецепты, группы, таблицы добычи и наименования объектов, а также официальная локализация. Эти данные доступны в машиночитаемом виде и позволяют построить базу знаний автоматически, без ручного наполнения. Часть сведений, однако, в файлах данных отсутствует и задаётся программным кодом среды; вытекающее из этого ограничение базы знаний рассматривается в подразделе 1.3.

Далее в пояснительной записке используется следующая терминология. Гибридная интеллектуальная система – система, в которой логико-символьные и нейросетевые компоненты объединены в единой модели решения задач. Решатель задач – компонент, выполняющий вывод на знаниях базы и формирующий решение. План – упорядоченная совокупность типизированных шагов, приводящая от текущего состояния пользователя к поставленной цели. Вето – отклонение решателем действия, предложенного языковой моделью, с указанием причины.

1.2 Анализ решаемого класса задач

Разрабатываемая система решает задачи трёх классов.

Планирование. По заданной цели и текущему состоянию пользователя требуется построить последовательность действий, приводящую к достижению цели. Условиями задачи являются целевой объект, требуемое количество и содержимое инвентаря; результатом – план с типизированными шагами либо обоснованное заключение о недостижимости цели. Задача относится к классической постановке планирования в пространстве состояний. Критерием корректности служит выполнимость каждого шага плана при условии выполнения предшествующих, критерием качества – трудоёмкость плана для пользователя. Частным случаем является декомпозиция дальней цели на последовательность посильных заданий, выполнимость каждого из которых доказывается от состояния, ожидаемого после выполнения предыдущих[5, 6].

Проверка допустимости. По предложенному действию требуется установить, разрешено ли оно правилами предметной области, и при отказе назвать причину. Условиями задачи являются вид действия, его объект и количество; результатом – вердикт с объяснением. Критерием корректности является отсутствие ложно разрешённых действий; отказ в действии, которое могло быть разрешено, считается менее тяжёлым нарушением, чем выполнение недопустимого.

Понимание запроса. По свободной реплике пользователя требуется определить намерение и сопоставить упомянутые объекты с каноническими идентификаторами предметной области. Результатом является структурированное представление запроса либо уточняющий вопрос, если запрос допускает несколько истолкований.

Совместное применение двух подходов необходимо потому, что ни один из них не решает всех трёх задач. Задача понимания запроса не поддаётся формальному описанию: множество способов выразить одно и то же намерение на естественном языке не перечислимо, а разговорные и жаргонные наименования объектов не содержатся ни в каких справочниках среды. Напротив, задачи планирования и проверки допустимости требуют гарантированной корректности, которую вероятностная модель обеспечить не может: она не располагает состоянием инвентаря, не выполняет вывода по графу способов получения и не проверяет своих утверждений[1, 7, 8].

Таким образом, распределение ролей определяется не соображениями удобства, а природой задач: нейросетевой компонент отвечает за понимание и формулирование, логико-символьный – за знание, вывод и гарантии.

1.3 Анализ базы знаний, решателя задач и интерфейса

Гибридная интеллектуальная система строится из трёх компонентов, каждый из которых решает самостоятельную задачу и может обновляться независимо от остальных. Интерфейс унаследован от предшествующего этапа курсового проектирования и расширен; база знаний и решатель задач разработаны заново, поскольку предметная область требует онтологического представления знаний и вывода по графу способов получения объектов. Ниже определяются назначение каждого компонента, используемые им знания, входные и выходные данные, а также накладываемые на него ограничения.

База знаний хранит сведения о предметной области в форме, пригодной для машинного вывода. Она содержит описания предметов, блоков и существ с их наименованиями, рецепты с составом и выходным количеством, группы взаимозаменяемых предметов, таблицы добычи и требования к инструменту. Знания представляются в виде онтологии в формате RDF/OWL, что обеспечивает явное задание отношений между сущностями и отделение знаний от программного кода. Словарь онтологии разделяется на две части: ядро, не зависящее от конкретной среды, и множество индивидов конкретной среды. Такое разделение позволяет подключить другую виртуальную среду наполнением второй части без изменения первой.

Входными данными базы знаний служат штатные файлы данных среды, из которых она собирается автоматически, и рукописный слой правил, описывающий ограничения, не выводимые из этих файлов. Выходными данными являются ответы на запросы о сущностях, их наименованиях, способах получения и наложенных ограничениях. Ограничением компонента выступает полнота исходных данных: сведения, задаваемые программным кодом среды, а не файлами, в базу знаний не попадают.

Решатель задач выполняет вывод на знаниях базы и формирует решение. Он включает планировщик, строящий план достижения цели обратным выводом от цели к имеющимся у пользователя ресурсам, и валидатор, проверяющий допустимость предложенного действия по правилам базы знаний. Входными данными решателя являются цель, требуемое количество, состояние пользователя и предложенное языковой моделью действие; выходными – план с типизированными шагами, вердикт о допустимости и цепочка применённых правил, образующая объяснение.

Существенным свойством решателя является право вето: вердикт о недопустимости действия не может быть изменён языковой моделью, поскольку исполняемая команда формируется после проверки. Решатель обращается только к словарю ядра онтологии и не содержит сведений о конкретной среде, что обеспечивает его переносимость. Ограничением компонента выступает вычислительная сложность вывода: граф способов получения объектов содержит взаимно зависимые узлы, поэтому перебор вариантов требует ограничения по глубине и ширине.

Интерфейс обеспечивает приём реплик пользователя и представление результатов. Он включает модификацию клиента виртуальной среды, принимающую реплики из игрового чата и отображающую ответы, внутриигровой экран для представления плана и объяснений, а также серверную часть, принимающую запросы и передающую команды на исполнение. Входными данными интерфейса являются реплика пользователя и телеметрия состояния: содержимое инвентаря, показатели здоровья, параметры окружения; выходными – ответ ассистента, дерево плана с отметками наличия ресурсов и исполняемая команда.

Ограничением интерфейса выступает требование к времени отклика: задержка между репликой и ответом не должна нарушать восприятие диалога как живого. Из этого следует необходимость разделения каналов представления: краткие ответы и уточняющие вопросы выводятся в чат, тогда как многошаговый план и цепочка объяснения – на отдельный экран, поскольку в строку чата они не помещаются.

Взаимодействие компонентов организуется так, что каждый из них решает задачу, соответствующую его природе. Языковая модель извлекает намерение из свободной реплики, база знаний предоставляет факты предметной области, решатель выполняет вывод и выносит вердикт, интерфейс обеспечивает представление результата. Замена любого из компонентов не должна требовать изменения остальных.

1.4 Анализ существующих гибридных интеллектуальных систем

Нейросимвольные архитектуры. Направление, объединяющее обучаемые модели с символьным представлением знаний, оформилось как самостоятельное и рассматривается в качестве третьей волны развития искусственного интеллекта. Типовым решением является разделение ответственности, при котором нейросетевая часть выполняет восприятие и выдвижение гипотез, а символьная – проверку, структурирование и объяснение. Преимуществом является проверяемость результата, ограничением – необходимость согласования представлений данных на границе компонентов. Подход применим к рассматриваемой задаче непосредственно и принят в качестве основного[2].

Архитектура RAG. Схема дополнения генерации извлечением предполагает поиск релевантных фрагментов во внешнем хранилище и передачу их языковой модели в качестве контекста. Достоинством является опора ответа на проверяемые источники, ограничением – то, что итоговый ответ по-прежнему порождается моделью и не подлежит формальной проверке. Для задач планирования и проверки допустимости, где требуется гарантия, схема недостаточна, однако её основной принцип – передача модели фактов из внешнего хранилища вместо знаний из весов – используется в разрабатываемой системе: ответ формулируется по выходу решателя[9].

Системы вызова инструментов. Подход, при котором языковая модель выбирает внешнюю функцию и её аргументы, а также схемы чередования рассуждения и действия обеспечивают выполнение моделью операций во внешней системе. Ограничением подхода является то, что корректность аргументов остаётся на ответственности модели. Разрабатываемая система использует этот механизм, однако аргументы подвергаются проверке решателем, а исполняемая команда формируется сервером[10, 11].

Агенты в виртуальных средах. Система Voyager демонстрирует обучаемого агента в среде Minecraft, накапливающего библиотеку навыков в виде программного кода. Платформа MineDojo предоставляет среду и набор задач для обучения агентов, а набор данных MineRL – записи действий человека для обучения с подкреплением. Общим для этих работ является сосредоточенность на автономном поведении агента, тогда как в настоящей работе решается иная задача – поддержка человека, формулирующего цели на естественном языке. Существенно также, что перечисленные системы не ставят задачи гарантированного соблюдения ограничений[3, 4, 12].

Экспертные системы и продукционные правила. Классический подход к представлению знаний в виде правил обеспечивает объяснимость вывода и проверяемость результата, но требует ручного наполнения базы и не работает со свободными формулировками. В разрабатываемой системе

продукционные правила применяются для описания ограничений, которые не выводятся из данных среды[13].

Семантические технологии. Технология OSTIS и языки описания онтологий предоставляют средства формального представления знаний предметной области и вывода на них. Онтологическое представление принято в работе в качестве формы хранения базы знаний, поскольку оно обеспечивает явное задание отношений между сущностями и независимость знаний от программного кода[14, 15].

Сопоставление рассмотренных подходов по трём свойствам, существенным для разрабатываемой системы, приведено в таблице 1.1.

Таблица 1.1 – Сопоставление подходов к построению интеллектуальных ассистентов

Подход	Свободная речь	Гарантии корректности	Учёт состояния
Продукционные правила	нет	да	да
Языковая модель	да	нет	нет
RAG	да	частично	нет
Вызов инструментов	да	нет	частично
Нейросимвольная схема	да	да	да

Ни один из подходов, кроме нейросимвольного, не обеспечивает одновременно всех трёх свойств, что и обосновывает выбор гибридной архитектуры.

1.5 Требования к гибридной интеллектуальной системе

К гибридной интеллектуальной системе предъявляются следующие функциональные требования:

– система должна решать задачи планирования достижения цели, проверки допустимости действий и понимания свободных запросов пользователя;

– система должна распознавать намерение из реплики, сформулированной произвольно, включая разговорные наименования объектов и их словоформы;

– система должна строить план достижения цели с учётом содержимого инвентаря пользователя, раскрывая изготовление, добычу и требования к инструменту, в том числе рекурсивно;

– система должна проверять предложенное действие по правилам базы знаний и отклонять недопустимое с указанием причины;

– система должна поддерживать два режима работы, различающиеся составом доступных действий;

– система должна формировать задания пользователю только после подтверждения их выполнимости выводом по базе знаний;

– система должна раскладывать дальнюю цель на последовательность заданий с наградами, доказывая выполнимость каждого задания от состояния, ожидаемого после предыдущих, и не допуская наград, обесценивающих последующие задания;

– система должна сопровождать решение объяснением, представляющим последовательность применённых правил;

– при неоднозначном запросе система должна задавать уточняющий вопрос, при некорректном – отказывать с объяснением, не выполняя молчаливой подстановки значений;

– при недостижимости цели система должна указывать шаг, на котором вывод оказался невозможен.

Нефункциональные требования к системе:

– корректность: доля недопустимых действий должна составлять ноль вне зависимости от формулировки запроса;

– полнота: план должен строиться для объектов, способы получения которых представлены в данных среды; границы применимости должны быть измерены и документированы;

– устойчивость: система должна сохранять работоспособность при некорректных, неполных и противоречивых запросах, а также при замене языковой модели на менее мощную;

– объяснимость: каждое решение должно сопровождаться воспроизводимой цепочкой применённых правил;

– производительность: время вывода решателя не должно превышать 200 мс на запрос, суммарное время отклика системы – 2 с;

– модульность: база знаний, решатель, интерфейс и языковая модель должны допускать независимую замену; решатель не должен содержать сведений о конкретной среде;

– воспроизводимость: база знаний должна собираться из исходных данных автоматически, результаты тестирования – воспроизводиться при фиксированных версиях моделей и параметрах.

1.6 Выводы

Предметной областью работы является взаимодействие пользователя с интерактивной виртуальной средой на естественном языке; в качестве среды выбрана Minecraft версии 1.20.1, обладающая машиночитаемым описанием своих объектов. Выделены сущности предметной области и отношения между ними, образующие граф способов получения объектов. Установ-

лено, что решение задач требует совместного использования знаний о мире и сведений о текущем состоянии пользователя.

Определены три класса решаемых задач: планирование, проверка допустимости и понимание запроса. Показано, что необходимость объединения двух подходов обусловлена природой этих задач: понимание свободной речи не поддаётся формализации, тогда как планирование и проверка требуют гарантированной корректности, недостижимой вероятностными методами.

Определён состав системы из трёх компонентов. База знаний хранит сведения о предметной области в виде онтологии, собираемой автоматически из данных среды, и дополняется рукописным слоем правил. Решатель задач выполняет вывод, строит план и выносит вердикт о допустимости действия, обладая правом вето и обращаясь только к словарю ядра онтологии. Интерфейс обеспечивает приём реплик, передачу телеметрии состояния и представление результатов по двум каналам: краткие ответы в чат, план и объяснения на отдельный экран.

Рассмотрены существующие решения: нейросимвольные архитектуры, схема RAG, системы вызова инструментов, агенты в виртуальных средах, экспертные системы и семантические технологии. Сопоставление показало, что одновременную поддержку свободной речи, гарантий корректности и учёта состояния обеспечивает только нейросимвольная схема, которая и принята в качестве основной.

Сформированы функциональные и нефункциональные требования к системе. Ключевым из них является требование нулевой доли недопустимых действий вне зависимости от формулировки запроса, достижимое лишь при вынесении проверок за пределы языковой модели.

2 ПРОЕКТИРОВАНИЕ ГИБРИДНОЙ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

2.1 Определение пользователей гибридной интеллектуальной системы

С гибридной интеллектуальной системой взаимодействуют пользователи трёх категорий и внешние подсистемы трёх видов. Категории различаются ролью, уровнем подготовки и полномочиями. Полномочия распределяются так, чтобы ни одна категория не могла получить действие в среде в обход правил базы знаний: такое действие не доступно ни пользователю, ни администратору, ни внешнему сервису.

Игрок является конечным пользователем системы. Его цель состоит в том, чтобы получать предметы, планы достижения целей и задания, обращаясь к ассистенту на естественном языке. Игрок знает предметную область, но не знает внутреннего устройства системы и формулирует запросы произвольно: разговорными наименованиями, в разных словоформах, с опущенными подробностями. Полномочия игрока определяются режимом работы: в творческом режиме ему доступны выдача и удаление предметов и призыв существ, в режиме выживания – только построение плана, получение заданий и прохождение кампании к дальней цели. Ожидаемым результатом взаимодействия является выполненное действие, план, задание, уточняющий вопрос либо отказ с указанием причины.

Администратор базы знаний обеспечивает актуальность знаний о предметной области. Он пересобирает базу знаний из данных среды при смене её версии и ведёт рукописный слой знаний, не выводимых из этих данных: явные запреты с причинами и разговорные наименования объектов. От администратора требуется знание предметной области и модели представления знаний. Его полномочия ограничены содержимым базы знаний: к исполнению действий в среде он отношения не имеет.

Администратор системы выбирает провайдера и версию языковой модели, задаёт параметры сервера и проводит оценку качества системы на размеченном наборе запросов. От него требуется знание устройства системы и её конфигурации. Изменение правил базы знаний и вердиктов решателя в его функции не входит.

Клиент виртуальной среды является внешней подсистемой, через которую пользователь взаимодействует с ассистентом. Он передаёт серверу реплики пользователя и телеметрию состояния, отображает ответы и исполняет команды. Полномочий на принятие решений клиент не имеет: исполняются только команды, сформированные сервером после проверки.

Провайдер языковой модели является внешним программным сервисом, выполняющим извлечение намерения из реплики и формулирование

ответа. Его ответ рассматривается системой как предложение, а не как решение: идентификаторы объектов и допустимость действия определяются логико-символьными компонентами, поэтому некорректный ответ модели не может привести к недопустимому действию.

Внешним источником данных служат штатные файлы данных среды и её официальная локализация. Из них база знаний собирается автоматически, без ручного переноса сведений.

2.2 Сценарии использования гибридной интеллектуальной системы

Варианты использования системы приведены на рисунке 2.1. Варианты игрока образуют основную часть: каждый из них реализуется сквозным процессом от реплики до результата. Варианты администраторов обеспечивают сопровождение системы и не связаны с обработкой запросов пользователя.

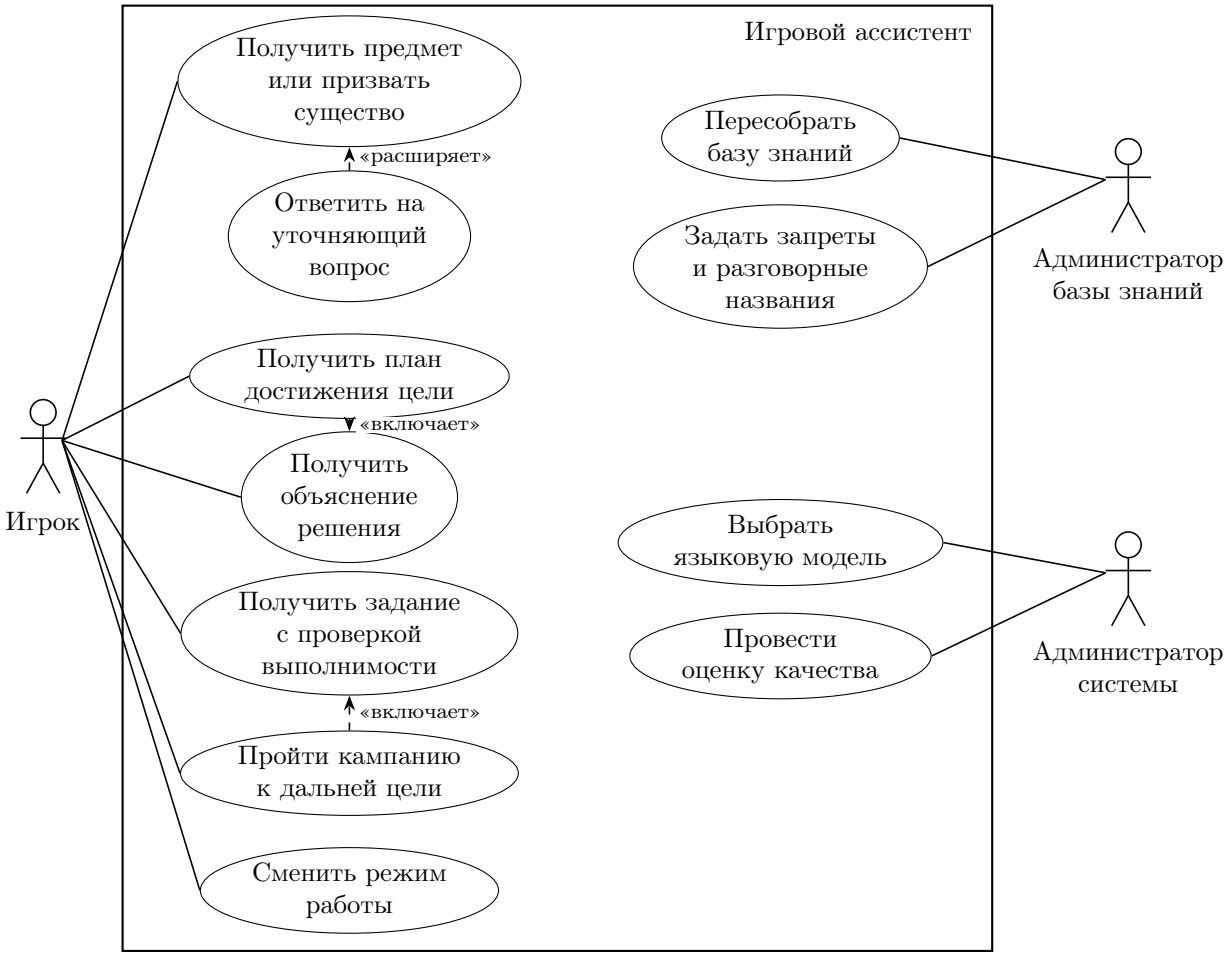


Рисунок 2.1 – Диаграмма вариантов использования

Ниже описываются сценарии, выделенные по характеру исхода: ти-

пичные, при которых запрос понят и выполнен; граничные, при которых запрос допускает несколько прочтений; ошибочные, при которых действие недопустимо или объект не существует. Для каждого сценария указываются инициатор, предусловия, входные данные, основной поток, альтернативные потоки и результат.

Сценарий 1. Выдача предмета по свободной реплике. Инициатором является игрок, предусловием – творческий режим работы. Входные данные: реплика вида «мне бы 10 наковален» и снимок состояния игрока. Основной поток включает следующие шаги:

– языковая модель извлекает из реплики действие, название объекта словами игрока и количество;

– количество, названное цифрами, сверяется с текстом реплики и при расхождении берётся из текста;

– резолвер сопоставляет название с сущностью онтологии;

– валидатор проверяет допустимость действия и формирует цепочку применённых правил;

– сервер собирает исполняемую команду из идентификатора, записанного в базе знаний;

– клиент исполняет команду, а пользователь получает подтверждение с итоговым количеством.

Альтернативный поток возникает при превышении предела количества: действие выполняется с предельным значением, которое обязательно называется в ответе. Результатом сценария является предмет в инвентаре игрока.

Сценарий 2. Построение плана достижения цели. Инициатором является игрок, предусловия отсутствуют: сценарий доступен в обоих режимах. Входные данные: реплика вида «как сделать алмазную кирку» и содержимое инвентаря. Основной поток совпадает с первыми четырьмя шагами сценария 1, после чего планировщик выполняет обратный вывод от цели к имеющимся ресурсам, раскрывая изготовление, добычу и требования к инструменту, и записывает рассмотренные варианты. Языковая модель формулирует ответ по выходу решателя, клиент отображает дерево плана на внутриигровом экране. Альтернативный поток возникает при недостижимости цели: план возвращается с отметкой шага, на котором вывод оказался невозможен. Результатом является план с отметками наличия ресурсов и объяснение выбора между вариантами.

Сценарий 3. Неоднозначный запрос. Инициатором является игрок. Входные данные: реплика вида «дай меч», допускающая несколько прочтений. Резолвер находит несколько сущностей с одинаковой оценкой соответствия и не выбирает ни одну из них. Пользователь получает уточняющий вопрос со списком вариантов, после ответа на который выполняется сценарий 1. Молчаливая подстановка одного из вариантов исключена: она дала

бы игроку не то, что он имел в виду, без какого-либо сигнала об этом.

Сценарий 4. Запрещённое действие. Инициатором является игрок. Входные данные: реплика вида «призови дракона края». Резолвер опознаёт сущность, валидатор находит в базе знаний явный запрет на её призыв и возвращает отказ с причиной, записанной в правиле. Существенно, что объект опознаётся, а не теряется: пользователь получает ответ «знаю и не выполню по такой-то причине», а не «такого не существует». Переформулировка запроса, в том числе с попыткой внушить модели снятие ограничений, исход не меняет, поскольку вердикт выносится после модели и без её участия.

Сценарий 5. Задание с проверкой выполнимости. Инициатором является игрок. Языковая модель предлагает цель задания и награду, решатель строит план достижения цели из текущего состояния игрока, проверяет награду теми же правилами, что и выдачу, и оценивает трудоёмкость. Задание сообщается пользователю только при успешном прохождении всех проверок; невыполнимое или несоразмерное отклоняется до того, как пользователь его увидит.

Сценарий 6. Кампания к дальней цели. Инициатором является игрок: цель он называет сам, например «веди меня к алмазной кирке», либо просит предложить её. Входные данные: реплика и содержимое инвентаря. Решатель строит план достижения цели и вырезает из него вехи – инструменты, оборудование и саму цель, – каждая из которых становится заданием с наградой; выполнимость каждого этапа доказывается от состояния, ожидаемого после предыдущих. Если цель предлагает ассистент, решатель отбирает достижимые цели, а языковая модель выбирает одну из них; выбор вне перечня не принимается. Поручения, завершение этапов и вручение наград сообщаются от лица персонажа, реплики которого модель формулирует строго по фактам решателя. Прогресс этапа считается из телеметрии инвентаря; завершение этапа и следующее поручение объявляются без обращения игрока. Альтернативный поток возникает, если цель недостижима либо этап не укладывается в допустимую трудоёмкость и после дробления: формируется отказ с указанием этапа и причины. Результатом сценария является пройденная цепочка этапов и цель в инвентаре игрока.

Ошибочные потоки, общие для всех сценариев, обрабатываются единообразно. Если объект не сопоставлен ни с одной сущностью базы знаний, формируется отказ с указанием, что такого объекта в среде нет. Если количество некорректно, например отрицательно или равно нулю, формируется отказ с указанием допустимого диапазона. Если провайдер языковой модели недоступен, запрос повторяется, а при исчерпании попыток пользователь получает сообщение о недоступности модели; такой исход не смешивается с отказом системы, поскольку решения по запросу не принималось.

2.3 Общая архитектура гибридной интеллектуальной системы

Архитектура системы приведена на рисунке 2.2. Система состоит из клиента виртуальной среды и сервера, взаимодействующих по протоколу HTTP с обменом данными в формате JSON. Сервер включает оркестратор, резолвер, решатель задач и базу знаний; языковая модель подключается как внешний сервис. База знаний собирается импортёром из данных среды независимо от работы сервера.

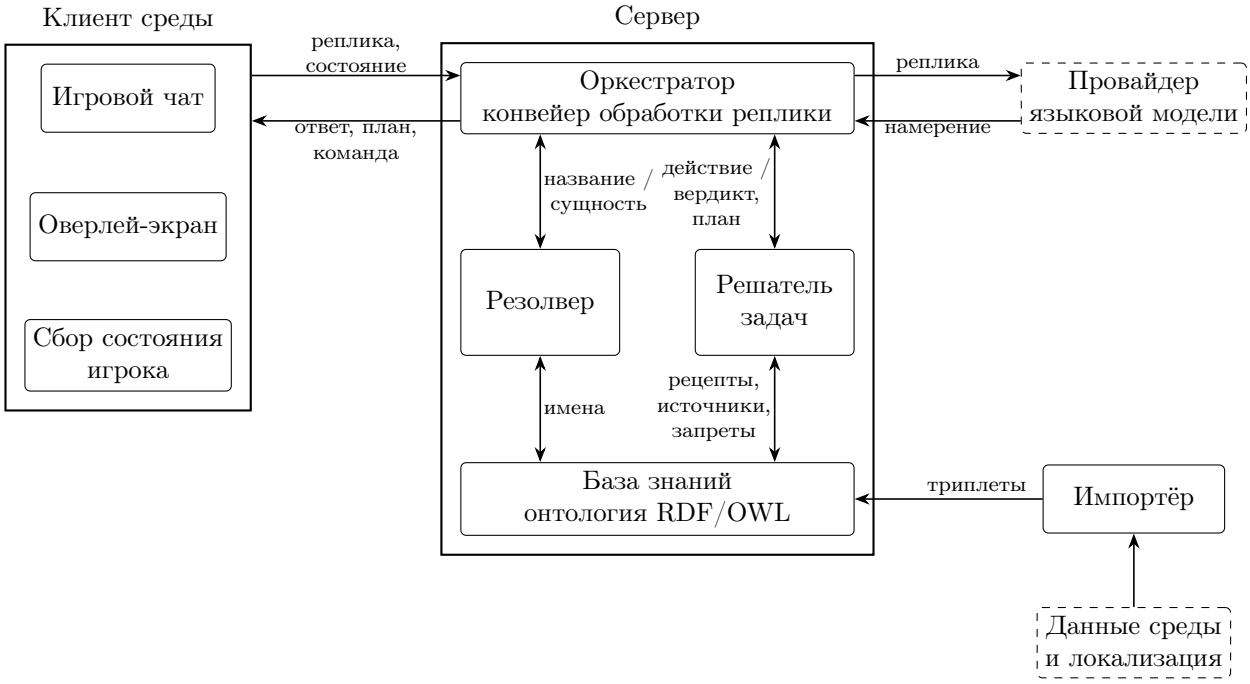


Рисунок 2.2 – Компонентная диаграмма гибридной интеллектуальной системы

Оркестратор реализует конвейер обработки реплики и является единственным компонентом, знающим порядок участия остальных. Порядок фиксирован: языковая модель определяет, что хочет пользователь; резолвер – о каком объекте идёт речь; валидатор – допустимо ли действие; планировщик – как достичь цели; сервер – что исполнить. Вынесение порядка в одно место обеспечивает выполнение главного архитектурного требования: решение о допустимости принимается после языковой модели и не зависит от её ответа.

Интеграция логико-символьного и нейробионического подходов организована по последовательной схеме, в которой нейросетевой компонент находится на входе и на выходе конвейера, а логико-символьный – в его середине. На входе модель преобразует свободную реплику в структурированное намерение; на выходе – выход решателя в связный ответ. Все решения, требующие знаний о предметной области или гарантий корректности,

принимаются между этими точками. В кампании модель участвует в тех же двух точках: на входе выбирает цель среди кандидатов, отобранных решателем, на выходе формулирует реплики персонажа по фактам решателя. Точки интеграции и форматы обмена данными приведены в таблице 2.1.

Таблица 2.1 – Точки интеграции компонентов и форматы обмена данными

Точка интеграции	Направление	Передаваемые данные
Разбор реплики	клиент → оркестратор	реплика, инвентарь, режим, параметры окружения
Извлечение намерения	модель → оркестратор	действие, название объекта словами пользователя, количество
Сопоставление	резолвер → оркестратор	идентификатор сущности либо список кандидатов, оценка соответствия
Проверка	валидатор → оркестратор	вердикт, причина отказа, цепочка применённых правил
Планирование	планировщик → оркестратор	дерево плана, недостижимые узлы, развилки с отвергнутыми вариантами
Кампания	решатель → оркестратор	цепочка этапов с наградами, прогноз состояния после каждого этапа
Формулирование ответа	оркестратор → модель	выход решателя, снимок состояния, окно диалога
Исполнение	оркестратор → клиент	текст ответа, дерево плана, команда

Ключевым проектным решением является то, что языковая модель называет объект словами пользователя, а не идентификатором. Идентификатор подставляет резолвер по онтологии, допустимость проверяет валидатор, исполняемую команду собирает сервер из идентификатора, записанного в базе знаний. Модели доступен только выбор действия из закрытого перечня, тогда как сама команда ей недоступна ни на одном шаге. Этим обеспечивается право вето решателя: недопустимое действие не может быть получено формулировкой запроса, поскольку в решении о допустимости модель не участвует[10, 11].

Последовательность взаимодействия компонентов при выполнении сквозного сценария построения плана приведена на рисунке 2.3. В сценарии участвуют все компоненты системы, и каждый из них решает задачу, соответствующую его природе.

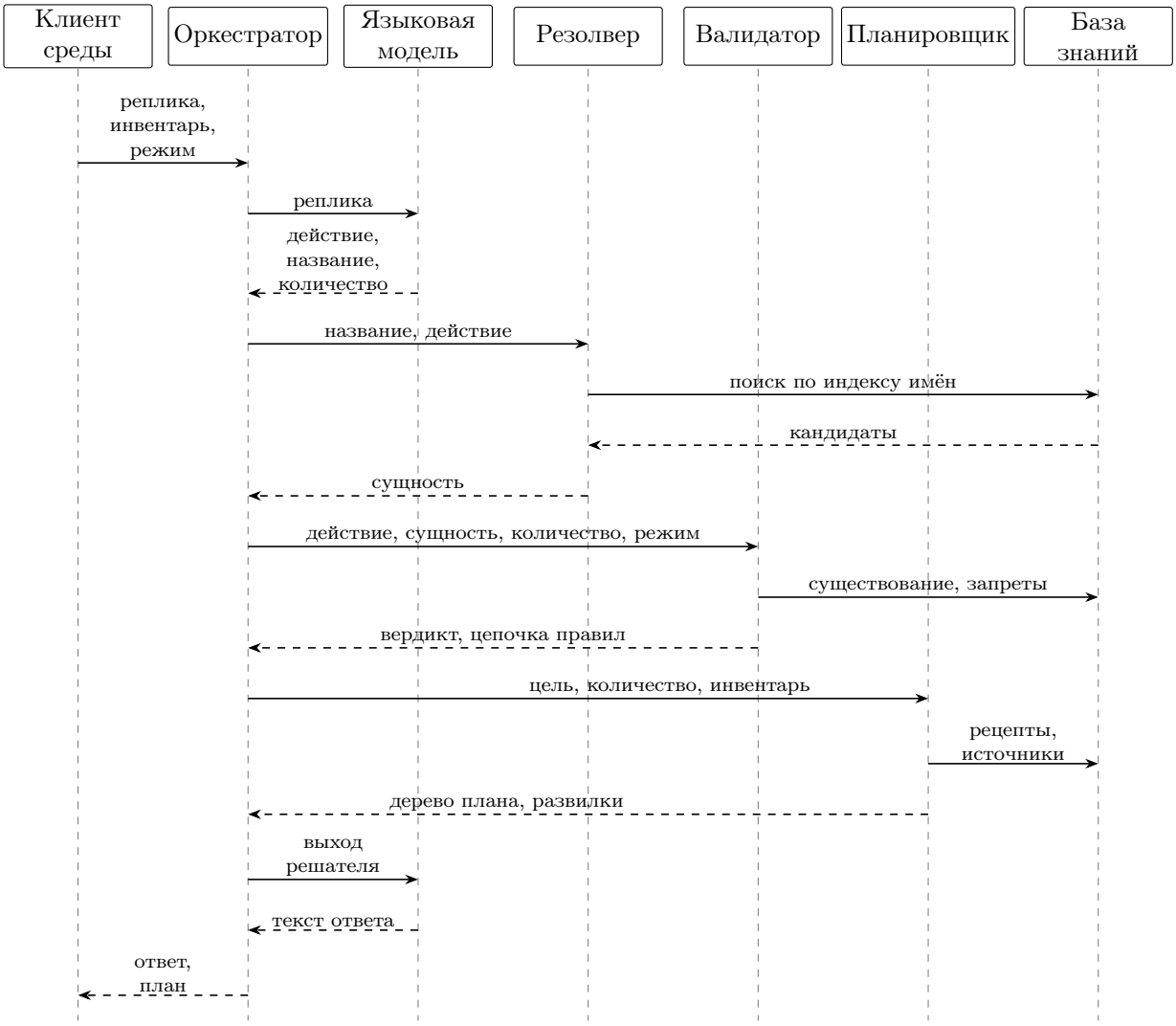


Рисунок 2.3 – Диаграмма последовательности сценария построения плана

Режим работы управляет составом доступных действий, а не текстом запроса к модели. В режиме выживания действие выдачи отсутствует в перечне, предъявляемом модели, и одновременно отклоняется валидатором. Двойная проверка исключает зависимость корректности от послушности модели: даже если модель предложит недоступное действие, оно не будет выполнено.

Контекст, передаваемый языковой модели, разделяется на три слоя по природе данных. Знания о предметной области в запрос к модели не помещаются никогда: модель получает к ним доступ только через выход решателя. Состояние пользователя передаётся компактным снимком, собираемым заново для каждого запроса. Из истории диалога передаются несколько последних реплик и сводка, которая формируется детерминированно из журнала выполненных действий, а не генерируется моделью. Такая организация снижает объём запроса и делает качество ответа слабо зависящим от мощности модели, поскольку модели не требуется ни помнить факты, ни восстанавливать состояние[9].

Архитектура обеспечивает независимое обновление компонентов. К базе знаний все компоненты обращаются через единый программный интерфейс и не зависят от формата её хранения. Решатель использует только словарь ядра онтологии и не содержит сведений о конкретной среде. Языковая модель заменяется изменением конфигурации без изменения кода. Клиент связан с сервером только форматом сообщений. Пересборка базы знаний при смене версии среды не затрагивает ни решатель, ни резолвер.

2.4 Проектирование алгоритмов и методов решения задач

Извлечение намерения. Задача решается языковой моделью, поскольку она требует понимания произвольной формулировки. Входными данными служит реплика пользователя, выходными – структура из трёх полей: действие из закрытого перечня, название объекта словами пользователя и количество. Формат ответа задаётся запросом к модели, генерация выполняется с нулевой температурой. Количество, названное в реплике цифрами, считывается регулярным выражением и имеет приоритет над количеством из ответа модели: модель пересказывает число, тогда как регулярное выражение его читает. Словесные количества остаются за моделью. При сбое канала связи запрос повторяется до трёх раз с нарастающей задержкой.

Сопоставление с онтологией. Задача состоит в определении сущности базы знаний по названию, данному пользователем. Резолвер строит при запуске обратный индекс, в котором каждой основе слова сопоставлены имена сущностей, её содержащие. Именами служат официальное название на русском языке, разговорные синонимы и английское название. Названия и запрос приводятся к нижнему регистру, из запроса удаляются

служебные слова и числительные, после чего слова заменяются основами по алгоритму Портера для русского языка. Основы, отсутствующие в индексе, сравниваются с основами словаря нечётко: при сходстве не ниже 0,8 совпадение засчитывается с понижающим коэффициентом 0,75. Этим закрываются беглые гласные, которые правила отсечения окончаний не обрабатывают.

Оценка соответствия имени запросу вычисляется по формуле

$$s = w \cdot c_N \cdot c_Q, \tag{2.1}$$

где w – вес имени, равный 1,0 для официального названия, 0,8 для синонима и 0,7 для английского названия; c_N – доля основ имени, найденных в запросе; c_Q – доля основ запроса, объяснённых именем. Произведение вместо суммы выбрано потому, что слабость по любой из сторон должна отбрасывать кандидата, а не усредняться со второй.

Решение резолвера трёхзначно. Кандидат отбрасывается, если объясняет менее 75 % слов запроса; этим запрос «дай звёздный меч», в котором слово «звёздный» не объясняет ни одно имя, отличается от запроса «дай меч», который любой из мечей объясняет целиком. Если кандидатов нет или оценка лучшего ниже 0,3, объект считается не найденным. Если отрыв лучшего кандидата от второго меньше 0,15, запрос признаётся неоднозначным, и пользователю предлагаются все близкие кандидаты. В остальных случаях сущность считается найденной. Синоним, слова которого целиком входят в официальное название другой сущности, обозначает класс объектов, а не одну сущность, и в индекс не включается: иначе синоним «меч», записанный когда-то за одним из мечей, превращал бы неоднозначный запрос в молчаливую подстановку.

Если лексический этап не нашёл ни одного кандидата, выполняется одна попытка переформулировки: языковая модель получает слова пользователя и возвращает официальное название объекта на английском и русском языках либо сообщает, что такого объекта в среде нет. Английское название запрашивается потому, что его модель воспроизводит устойчивее русского перевода. Возвращённые названия проходят тот же резолвер, поэтому модель не может создать несуществующий объект. Найденный кандидат никогда не приводит к действию напрямую: он предлагается пользователю в уточняющем вопросе, и действие выполняется только после подтверждения.

Для этой роли рассматривался также векторный этап: сравнение запроса с заранее рассчитанными векторами имён по косинусной близости. Замер на двух многоязычных моделях векторных представлений предложений показал, что близость не разделяет парафразы существующих объектов и названия несуществующих: у одной из моделей запрос «кракен»

получил близость 0,94 к верстаку, тогда как верная парафраза «железная броня на грудь» – 0,80 к нагруднику. Порога, отделяющего верные варианты от ложных, не существует, поэтому векторный этап отклонён. Переформулировка моделью на том же наборе нашла все верные парафразы и не предложила ни одного кандидата для несуществующих объектов.

Проверка допустимости действия. Валидатор выполняет проверки в фиксированном порядке и записывает исход каждой в цепочку, из которой формируется объяснение:

– доступность действия в текущем режиме;

– наличие цели действия;

– существование цели в базе знаний;

– отсутствие явного запрета на действие с целью;

– соответствие вида цели действию: выдать можно предмет или блок, призвать – только существо;

– корректность количества и его соответствие пределу.

Явный запрет проверяется раньше вида цели, поскольку ответ «запрещено правилом по такой-то причине» содержательнее ответа «это нельзя взять в инвентарь», даже когда верны оба. Первая же неудачная проверка завершает работу валидатора с отказом либо с уточняющим вопросом, если цель не указана. Единственное допустимое изменение запроса – уменьшение количества до предела – не является молчаливым: итоговое число обязательно называется в ответе.

Построение плана. Планировщик выполняет обратный вывод от цели к ресурсам, имеющимся у пользователя[5, 6]. Схема алгоритма вывода одного узла плана приведена на рисунке 2.4. Алгоритм вызывается рекурсивно для каждого ингредиента рецепта и для недостающего инструмента добычи.

На каждом узле сначала используется то, что уже есть в запасах, и только остаток потребности раскрывается дальше. Изготовление и добыча рассматриваются как равноправные способы, каждый на своей копии запасов, и выбирается лучший, а не первый сработавший. Варианты сравниваются лексикографически: сначала по выполнимости, затем по числу недостижимых узлов, затем по трудоёмкости. Трудоёмкость складывается из весов шагов: шаг изготовления или добычи весит единицу, каждый уровень требуемого инструмента добавляет два, охота на существо – двадцать, ненадёжное выпадение – тридцать. Веса являются проектным решением о том, что удобнее пользователю, и вынесены в именованные константы. Оценка рецепта включает цену оборудования, которого у пользователя нет: без этого выбор между печью и доменной печью определялся бы порядком записи, тогда как доменная печь сама требует печи. Рецепты, собираемые в инвентаре без верстака, помечены в базе знаний отдельно и оборудования не требуют.

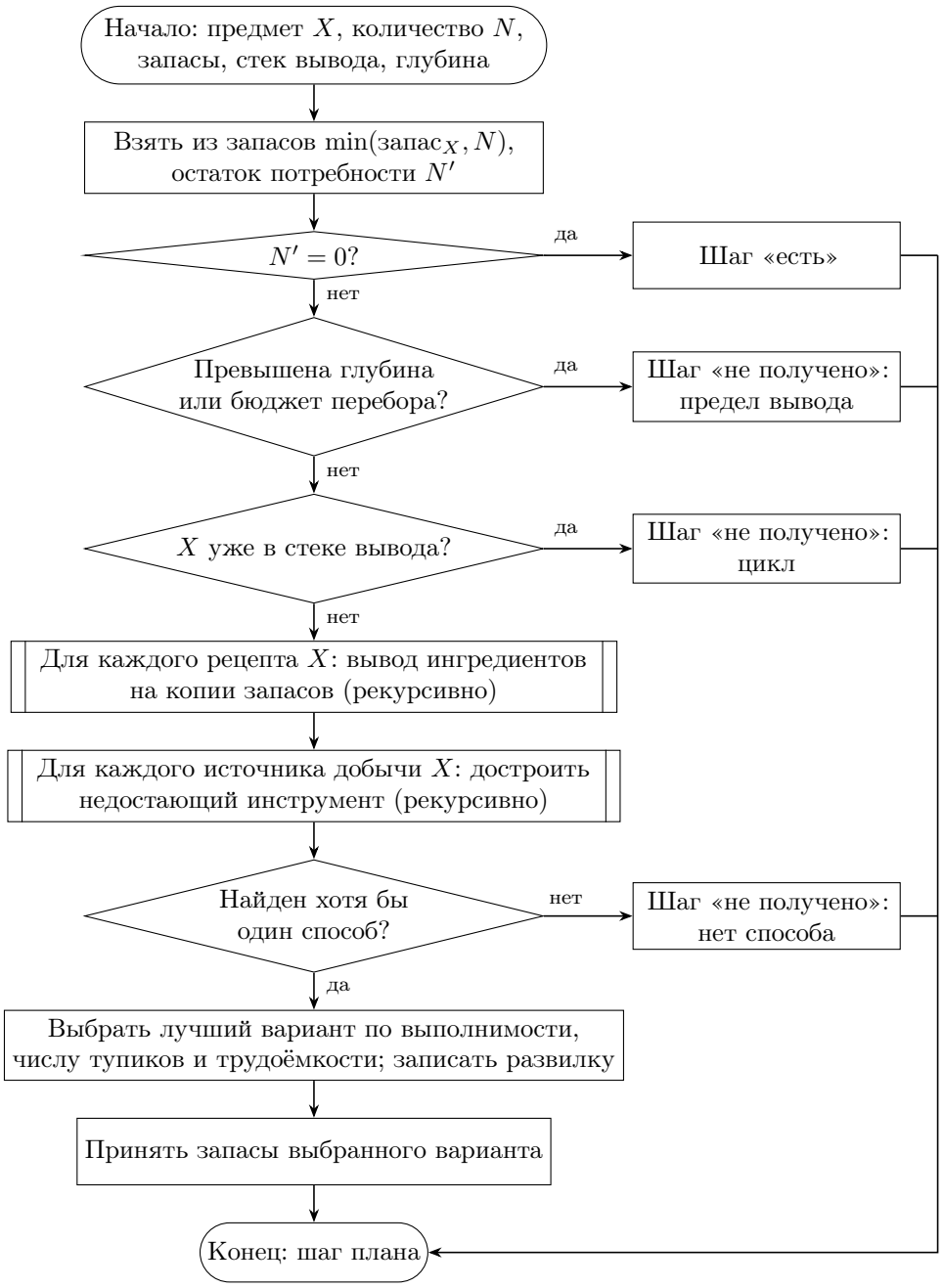


Рисунок 2.4 – Схема алгоритма вывода узла плана

Каждая развилка, на которой рассматривалось более одного способа, записывается вместе с отвергнутыми вариантами и их трудоёмкостью. Это позволяет решателю самому ответить на вопрос, почему план построен именно так: например, железный слиток можно получить, победив железного голема, но такой путь имеет трудоёмкость 21 против 17 у выплавки из руды и поэтому отвергается.

Проверка выполнимости задания. Задание, предложенное моделью, принимается только при выполнении трёх условий. Цель должна быть достижима из текущего состояния пользователя: для неё строится план. Награда должна проходить те же проверки, что и выдача предмета. Трудоёмкость задания, равная трудоёмкости плана с поправкой на объём цели, должна лежать в пределах от 1 до 24: задание короче нижней границы не ощущается достижением, длиннее верхней – обесценивает награду.

Декомпозиция цели на этапы. Кампания строится из дерева плана, полученного для дальней цели. Вехами становятся узлы, открывающие новую возможность: инструмент, требуемый для добычи (каменная кирка открывает железную руду), оборудование, требуемое рецептом (печь открывает переплавку), и сама цель. Обход дерева снизу вверх даёт этапы в порядке зависимостей; оборудование ставится перед шагом, который на нём выполняется; узлы, уже имеющиеся в инвентаре, вехами не становятся. Каждый этап проверяется теми же условиями, что одиночное задание, но от прогнозируемого состояния: к инвентарю добавляются цели пройденных этапов и вычитается взятое их планами, а излишки рецептов не засчитываются. Прогноз занижен намеренно: недооценка даёт лишнюю работу на следующем этапе, но не ложное доказательство. Этап, превысивший предел трудоёмкости, дробится: самое тяжёлое поддерево материала выделяется в предшествующий этап, и проверка повторяется; этап, не уложившийся и после дробления, делает кампанию невыполнимой с указанием причины.

Награда этапа выбирается из перечня расходуемых предметов по правилу: она не является целью кампании или этапа, не встречается в планах последующих этапов и, если это инструмент, не превосходит уровень, который будет у пользователя после этапа; количество растёт с трудоёмкостью в пределах лимита выдачи. Решатель отдаёт весь допустимый набор, из которого выбирается награда, уместная сюжету; выйти за пределы набора нельзя. Для предложения цели решатель перечисляет инструменты на ступень выше имеющихся по каждому виду и недостающее оборудование, оставляя те, для которых кампания строится не более чем из пяти этапов; языковая модель выбирает среди них, а выбор вне перечня заменяется кратчайшей из доказанных целей.

Формирование ответа. Ответ пользователю формулируется языковой моделью по выходу решателя: вердикту, плану, причине отказа. Факты в ответ попадают только из этого выхода, поэтому модель не может сообщить

пользователю того, чего не вывел решатель. Для запросов, не требующих действия, используется отдельный короткий запрос к модели без перечня действий и без обращения к базе знаний. Реплики кампании формулируются от лица персонажа, заданного пресетом: именем, родом занятий, манерой речи и обращением к игроку. Модель получает факты этапа и сводку кампании; реплика принимается, только если в ней названы этап и награда, иначе используется шаблон из тех же фактов.

Вычислительная сложность. Построение индекса резолвера выполняется один раз и линейно по суммарной длине имён. Поиск по точным основам требует обращения к индексу для каждого из k слов запроса; нечёткое сравнение выполняется только для основ, не найденных в индексе, и линейно по объёму словаря V, составляющему около 1,4 тыс. основ. Сравнение ограничивается основами близкой длины, что отсекает большую часть словаря до вычисления сходства. Валидатор выполняет постоянное число обращений к базе знаний.

Сложность планировщика в худшем случае экспоненциальна по глубине вывода: при ширине ветвления b и глубине d перебор составляет $O(b^d)$ узлов. Для удержания времени в пределах требования введены три ограничения: глубина вывода не более 20, не более четырёх вариантов рецепта на узел и не более 4000 раскрытых узлов на запрос. Варианты рецептов упорядочиваются по заранее рассчитанной оценке стоимости, так что при ограничении ширины рассматриваются наиболее дешёвые. Оценка рассчитывается один раз для всей базы знаний итеративной релаксацией: начальные значения задаются для объектов, добываемых в среде, и на каждом проходе уточняются для объектов, получаемых из них изготовлением, пока значения не перестанут меняться. Рекурсивный расчёт здесь неприменим, поскольку граф способов получения содержит циклы; релаксация нечувствительна к циклам и требует $O(I \cdot R)$ операций, где I – число проходов, не более 16, а R – число рецептов.

Объём базы знаний влияет на время предварительных расчётов линейно, а на время обработки отдельного запроса – через ширину ветвления, которая ограничена. Основной вклад во время отклика вносит обращение к языковой модели, поэтому запрос к ней минимизируется: в него не помещаются знания о предметной области, а разбор реплики и формулирование ответа выполняются отдельными короткими обращениями. Индексы рецептов, источников и имён строятся один раз на процесс и используются всеми запросами.

2.5 Выводы

В результате проектирования определены пользователи системы и внешние подсистемы с разграничением полномочий, при котором действие

в среде не может быть получено в обход правил базы знаний. Описаны сценарии использования: типичные, граничные и ошибочные, а также единообразная обработка ошибочных потоков.

Спроектирована архитектура с последовательной схемой интеграции, в которой языковая модель находится на входе и выходе конвейера, а решения о допустимости и построение плана принимаются логико-символьными компонентами между этими точками. Определены точки интеграции и форматы обмена данными, разделение контекста модели на три слоя и механизмы, обеспечивающие независимое обновление компонентов.

Спроектированы алгоритмы извлечения намерения, сопоставления с онтологией, проверки допустимости, построения плана, проверки выполнимости заданий, декомпозиции дальней цели на этапы с поэтапным доказательством выполнимости и отбором наград, а также формирования ответа. Проанализирована их вычислительная сложность и введены ограничения, удерживающие время вывода в пределах требований: ограничение глубины и ширины перебора, бюджет раскрываемых узлов, предварительная оценка стоимостей и индексация знаний.

3 РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И ДЕМОНСТРАЦИЯ ГИБРИДНОЙ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

3.1 Выбор средств и технологий реализации

3.2 Структура программной реализации

3.3 Реализация пользовательского интерфейса

3.4 Тестирование интеллектуальной системы

3.5 Оценка качества, эффективности и объяснимости системы

3.6 Демонстрация работы гибридной интеллектуальной системы

3.7 Анализ ограничений и направления развития

3.8 Выводы

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Survey of Hallucination in Natural Language Generation / Ziwei Ji [et al.] // ACM Computing Surveys. — 2023. — Vol. 55, no. 12. — Pp. 1–38.

[2] d'Avila Garcez, Artur. Neurosymbolic AI: the 3rd Wave / Artur d'Avila Garcez, Luis C. Lamb // Artificial Intelligence Review. — 2023. — Vol. 56, no. 11. — Pp. 12387–12406.

[3] Voyager: An Open-Ended Embodied Agent with Large Language Models / Guanzhi Wang [et al.] // Transactions on Machine Learning Research. — 2024.

[4] MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge / Linxi Fan [et al.] // Advances in Neural Information Processing Systems (NeurIPS 2022). — 2022.

[5] Fikes, Richard E. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving / Richard E. Fikes, Nils J. Nilsson // Artificial Intelligence. — 1971. — Vol. 2, no. 3-4. — Pp. 189–208.

[6] Ghallab, Malik. Automated Planning: Theory and Practice / Malik Ghallab, Dana Nau, Paolo Traverso. — San Francisco : Morgan Kaufmann, 2004. — 635 P.

[7] Рассел, С. Искусственный интеллект: современный подход / С. Рассел, П. Норвиг. — 4-е изд. — Москва : Вильямс, 2021. — 1408 с.

[8] Jurafsky, Daniel. Speech and Language Processing / Daniel Jurafsky, James H. Martin. — 3rd edition. — Stanford : Stanford University, 2024. — 636 P.

[9] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / Patrick Lewis [et al.] // Advances in Neural Information Processing Systems (NeurIPS 2020). — 2020. — Pp. 9459–9474.

[10] Toolformer: Language Models Can Teach Themselves to Use Tools / Timo Schick [et al.] // Advances in Neural Information Processing Systems (NeurIPS 2023). — 2023.

[11] ReAct: Synergizing Reasoning and Acting in Language Models / Shunyu Yao [et al.] // International Conference on Learning Representations (ICLR 2023). — 2023.

[12] MineRL: A Large-Scale Dataset of Minecraft Demonstrations / William H. Guss [et al.] // Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI 2019). — 2019. — Pp. 2442–2448.

[13] Джарратано, Дж. Экспертные системы: принципы разработки и программирование / Дж. Джарратано, Г. Райли. — 4-е изд. — Москва : Вильямс, 2006. — 1152 с.

[14] Голенков, В. В. Открытая технология онтологического проектирования, производства и эксплуатации семантически совместимых гибридных интеллектуальных компьютерных систем / В. В. Голенков, Н. А. Гулякина, Д. В. Шункевич. — Минск : Бестпринт, 2021. — 690 с.

[15] A Semantic Web Primer / Grigoris Antoniou, Paul Groth, Frank van Harmelen, Rinke Hoekstra. — 3rd edition. — Cambridge, MA : MIT Press, 2012. — 288 P.